

Multivariate Time-Series Anomaly Detection

Complete Project Work Summary

Prepared as a reference for team presentation - covers all work performed to date

1. Project Overview

The goal is to detect anomalies in multivariate sensor time-series data from an industrial process. Each raw file is one independent recording run (~18 sensor channels). Training data carries no anomaly labels; only a small validation set (10 runs) has ground truth; the final test set (53 runs) is unlabeled and scored externally by a competition portal (Codabench-style). Because training data is unlabeled, the core modeling approach is unsupervised: learn what “normal” sensor behavior looks like, and flag deviations from it.

2. Modeling Approach

Two independent detectors are combined into a hybrid ensemble:

- **CNN autoencoder** - learns to reconstruct short 200-timestep windows of sensor readings. Trained only on data that looks normal, so it reconstructs normal patterns well and anomalous ones poorly. Reconstruction error is the anomaly score. Captures *temporal* anomalies (slow drift, sustained abnormal patterns).
- **Isolation Forest** - a statistical outlier detector run on per-window summary statistics (mean, std, min, max, median, two percentiles) per sensor. Captures *statistical* outliers independent of temporal shape.
- **Hybrid fusion** - both scores are normalized to [0,1] and combined with a weight chosen on the validation set (not fixed by hand). A single threshold, also chosen on validation, converts the fused score into a binary anomaly prediction per timestep.

3. State of the Project Before This Work

An initial repository audit found that essentially all real model logic lived only inside Jupyter notebooks, copy-pasted 3-5 times across them with small, undocumented drift between copies. The intended src/ Python package was mostly empty stub files (0 bytes). Several concrete bugs were identified: a broken import from an incomplete refactor, a shadowed import, an off-by-one in threshold selection, an Isolation Forest feature bug that collapsed all sensors into a single scalar, windows that spliced across unrelated runs, and a CNN epoch count that had been silently reverted from a tuned value of 50 back down to 10. Reported baseline performance at that point (window-level granularity, not directly comparable to later per-timestep numbers) was:

Model	F1 (baseline, window-level)
CNN Autoencoder	0.60
Isolation Forest	0.40
Hybrid	0.62

4. Round 1 - Repository Cleanup and Critical Bug Fixes

4.1 Cleanup

- Populated the empty src/ stubs with real, deduplicated logic extracted from the notebooks: shared windowing utilities, the CNN model/train/predict modules, Isolation Forest feature/train/predict modules, shared evaluation metrics, and three runnable end-to-end pipeline scripts (including a new run_hybrid.py that did not exist before, even as a stub).
- Fixed the broken import and the shadowed-import bug so the codebase actually runs outside notebooks.
- Removed a byte-identical duplicate output directory, added the missing torch dependency, corrected swapped dataset counts in the data documentation, and deduplicated two notebooks that each contained multiple competing copies of the same pipeline.

4.2 Bug fixes affecting model quality

- **Isolation Forest per-sensor features:** window statistics were being computed across all 18 sensors flattened into one scalar (no axis argument) - fixed to compute per-sensor, giving the model real signal about which sensor was anomalous.
- **Threshold off-by-one:** threshold selection was misindexed against scikit-learn's precision-recall curve output, so the threshold actually applied did not match the F1 score being reported as “best.”
- **Run-boundary window leakage:** windows were built by concatenating every file in a split before slicing, splicing one run's tail onto an unrelated run's head - fixed to window each run separately.
- **Coarse scoring granularity:** validation/test were scored at one label per ~10-timestep block; rebuilt to score densely (every timestep) and average overlapping windows, matching the true submission format instead of a coarser proxy for it.
- **CNN epoch count** restored to 50 (the value the project's own tuning notebook had already proven necessary for convergence, but which had been silently reverted to 10 in the pipeline that actually produced submissions).
- **Fusion weight** (previously hardcoded 0.85/0.15) is now chosen by sweeping against validation.

4.3 Round 1 Results (validation set, per-timestep - a stricter metric than the baseline table above)

Model	F1	Precision	Recall	ROC-AUC
Isolation Forest (tabular)	0.437	0.30	0.83	0.585
Isolation Forest (window, in hybrid)	0.491	0.33	0.92	0.649
CNN autoencoder (standalone)	0.408	0.30	0.64	0.574
Hybrid (fused)	0.494	0.34	0.92	0.636

The hybrid predictions were submitted to the real competition portal: **public F1 = 0.399, private F1 = 0.471** - reasonably close to the local validation estimate (0.494), which confirmed the local evaluation methodology is trustworthy and not badly miscalibrated.

5. Round 2 - Targeted Model-Quality Improvements

- **CNN checkpoint selection:** added validation-based early stopping (evaluate every 5 epochs during training, keep the best-scoring checkpoint) after observing that the training loss fell smoothly for all 50 epochs with no plateau - a classic overtraining signature. Result: standalone CNN F1 improved from 0.408 to **0.458** (best checkpoint at epoch 5), directly confirming the overtraining hypothesis.
- **Isolation Forest contamination sweep:** swept the contamination hyperparameter (0.1-0.3), which had been hand-set and never tuned. Result: all five candidates produced the identical F1 (0.4907) - a confirmed, useful negative finding. Contamination only affects the model's own internal classification cutoff; since the pipeline scores with the continuous decision function and does its own threshold search, this hyperparameter has zero effect here.
- **Finer fusion weight grid:** widened from four coarse candidates to nine (0.1 steps) - free to compute since both branches' scores are already available.

Model	Round 1 F1	Round 2 F1	Change
CNN (standalone)	0.408	0.458	+0.050
Isolation Forest (window)	0.491	0.491	unchanged
Hybrid (fused)	0.494	0.494	+0.0006 (flat)

Despite the CNN's real standalone gain, the hybrid barely moved, and the optimal fusion weight actually shifted further toward Isolation Forest (0.4 CNN / 0.6 ISO vs. 0.5/0.5 before) - improving one branch's own accuracy did not make it a better ensemble partner, suggesting the two branches' errors overlap more than the hybrid design assumed.

6. Root Cause Analysis - Why Performance Plateaued

A full technical post-mortem was written up separately (see reports/hybrid_model_postmortem.pdf); summary of the key findings:

- Both branches show weak-but-real score separability (ROC-AUC 0.61-0.65, where 0.5 is random and 1.0 is perfect) - every threshold/calibration bug identified was fixed, and F1 stayed in the same 0.40-0.49 band regardless, which is the signature of a signal ceiling in the model/features, not a remaining bug.
- The CNN's architecture was a single shallow Conv1d block with an effective receptive field of only ~3 timesteps out of a 200-step window, despite the project's own tuning experiments already proving anomalies here are slow, sustained drifts that reward wider temporal context.
- Training data is unlabeled and not guaranteed anomaly-free; the CNN's early-peak-then-degrade pattern (Round 2) is consistent with the autoencoder gradually learning to reconstruct some anomalous patterns too, shrinking the score gap the whole approach depends on.
- At the F1-optimal threshold, the model over-predicts heavily: roughly 65-88% of timesteps flagged anomalous against a true validation rate of 24%, and 39-40 of 53 test runs predicted as entirely anomalous end to end - a direct symptom of weak separability, not a threshold-search mistake.

7. Round 3 - Dilated CNN Architecture

The most directly actionable root cause is the CNN's shallow receptive field. The architecture was upgraded from a single Conv1d block to a stacked **dilated residual (TCN-style) architecture** (dilation rates 1, 2, 4, 8), expanding the effective receptive field from ~3 timesteps to ~63 before the bottleneck, without enlarging the window or increasing memory cost the way a wider window would.

LSTM and Transformer alternatives were considered and set aside for this round: LSTMs are slower and less stable to train on 200-step sequences; Transformers are data-hungry and add more tuning risk than is justified given this project's data volume and timeline. The dilated CNN was chosen as the closest, lowest-risk extension of the existing architecture.

Metric	Round 2 (old CNN)	Round 3 (dilated CNN)
CNN standalone F1	0.458	0.484
CNN standalone precision	0.324	0.347
CNN standalone recall	0.777	0.796
CNN standalone ROC-AUC	0.614	0.605
Isolation Forest F1 (unaffected)	0.491	0.491
Hybrid (fused) F1	0.494	0.494
Predicted anomaly rate	0.656	0.894
Test runs predicted 100% anomalous	39 / 53	41 / 53

Result: mixed. The dilated architecture delivered a real, reproducible standalone CNN gain (F1 0.458 to 0.484, precision 0.324 to 0.347), and training no longer shows the same early-peak-then-degrade overtraining

pattern - validation F1 trended upward through epoch 35 rather than peaking at epoch 5. **However, the fused hybrid F1 did not move at all** (0.494 to 0.494, identical to four significant figures), and the over-prediction problem got marginally worse, not better (41 of 53 test runs now predicted fully anomalous, vs. 39 before). This is the same pattern seen in Round 2: improving the CNN branch's own accuracy does not improve the hybrid, because Isolation Forest's ~0.49 F1 is acting as a hard ceiling on the fusion regardless of CNN quality - the two branches' errors appear to overlap more than the hybrid design assumes. The architecture upgrade was a genuine, worthwhile fix on its own terms, but it confirms the actual bottleneck for the number that matters (hybrid F1) is now the fusion itself and/or Isolation Forest's feature ceiling, not CNN capacity.

8. Infrastructure Work

- Set up SSH key-based access to RPTU's "Elwetritsch" HPC cluster for GPU-accelerated training.
- Diagnosed and worked around a VPN routing conflict between the university network and the tool used to access this assistant.
- Wrote a SLURM batch script (scripts/run_hybrid.slurm) requesting a V100 GPU, loading the correct CUDA module (12.8, matched to PyTorch's officially supported build), and running the hybrid pipeline end-to-end on the cluster.
- All code is version-controlled and pushed to GitHub (purnanandiganaa-alt/ts-anomaly-detection), so the cluster run and local development stay in sync.

9. Summary Results Across All Rounds

Stage	CNN F1	Isolation Forest F1	Hybrid F1	Notes
Original baseline	0.60	0.40	0.62	window-level metric
Round 1 (bug fixes)	0.408	0.491	0.494	per-timestep metric; portal: public 0.399 / private 0.471
Round 2 (checkpoint sel.)	0.458	0.491	0.494	CNN gained, hybrid flat
Round 3 (dilated CNN)	0.484	0.491	0.494	CNN gained again, hybrid still flat

10. Key Takeaways

- The original repository had all real logic trapped in notebooks with no reusable, tested implementation - this was fixed first, before any modeling changes, so every subsequent result is reproducible via a single command.
- Several genuine correctness bugs (not just tuning gaps) were found and fixed; some had a large, measurable impact (Isolation Forest's feature bug), others confirmed a hyperparameter didn't matter at all (contamination), which is itself a useful, time-saving finding.
- The CNN branch improved twice in a row (0.408 to 0.458 to 0.484) through checkpoint selection and then a deeper dilated architecture - both real, reproducible, well-understood wins on their own terms.
- Despite that, hybrid F1 has been essentially flat (0.494, 0.494, 0.494) across both rounds of CNN improvement. The bottleneck for the number that actually matters is no longer the CNN - it is Isolation

Forest's ceiling and/or the fusion mechanism itself, which is the clearest, most important finding from this round of work.

- Local validation-based evaluation has been shown to track real competition-portal scores reasonably closely, giving confidence that further local improvements should genuinely transfer.

11. Next Steps

- **Redirect effort away from the CNN branch** - two rounds of CNN-only improvement have not moved hybrid F1 at all, so further CNN tuning is unlikely to be the highest-leverage next step.
- Investigate why Isolation Forest and the CNN produce correlated rather than complementary errors - richer/cross-sensor window features for Isolation Forest, or a learned (not grid-searched) fusion combiner, are both more likely to move the hybrid number than further CNN architecture work.
- Investigate contamination-robust training (down-weighting high-error windows during training) as a complementary fix, since it addresses a different failure mode (training-set contamination) than architecture depth, and affects both branches since both train on the same unlabeled data.
- Re-submit the current best predictions to the competition portal for a real test-set readout, since the hybrid prediction file has changed even though its validation F1 has not.